

DSA STUDY BLUEPRINT SERIES

11. Search in Rotated Sorted Array

Searching in a Pivot-Rotated Sorted Array

Section 02 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Pivot Rotation:** Array is rotated at some pivot point.
- **Logic:** At least one half of the array (left or right of mid) is always sorted.
- Identify the sorted half, then check if target is inside its bounds.

Memory & Execution Blueprint



C++ Implementation Reference

Standard code layout and syntax

```
int searchRotated(vector& nums, int target) {
    int s = 0, e = nums.size() - 1;
    while (s <= e) {
        int mid = s + (e - s) / 2;
        if (nums[mid] == target) return mid;
        if (nums[s] <= nums[mid]) {
            if (target >= nums[s] && target < nums[mid]) e = mid - 1;
            else s = mid + 1;
        } else {
            if (target > nums[mid] && target <= nums[e]) s = mid + 1;
            else e = mid - 1;
        }
    }
    return -1;
}
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(\log N)$
Space Complexity	$O(1)$

Key Practices & Gotchas

- **Important:** If array has duplicates, worst case falls back to $O(N)$ linear time.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's dry run Binary Search looking for target 47 in [12, 24, 35, 47, 58, 69, 80] :

Iteration	Search Range Indices	Mid Index Calculation	Element at Mid	Comparison & Action
1	[0, 6]	$mid = 0 + (6-0)/2 = 3$	<code>arr[3] = 47</code>	<code>47 == 47</code> (Target found at index 3!)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Lower/Upper Bounds:** Binary search variations resolve the first element not-less-than target (lower bound) or strictly-greater-than target (upper bound).
- **Search Space Boundary:** Ensure checking `start ≤ end` conditions and compute mid indices as `s + (e-s)/2` to prevent integer overflow.
- **Duplicates:** Adjust left/right boundaries inwards to trace duplicate values edges.
- **Related LeetCode Problems:** #34 Find First and Last Position of Element in Sorted Array, #704 Binary Search, #33 Search in Rotated Sorted Array.